



Dynamic Traffic Optimization in Real-Time Routing Systems

Vansh Bansal (2024csb1167) ,
Telgamalla Gowtham (2024csb1165) ,
Tejas Singh (2024csb1164)

Instructor:
Dr. Apurva Mudgal

Summary: This project presents a comprehensive real-time traffic routing system that integrates multiple pathfinding algorithms — A* Search, Dijkstra’s Algorithm, and Breadth-First Search (BFS) — within a unified framework. The system loads road network graphs from CSV files, simulates dynamic traffic conditions across normal, peak-hour, and heavy-traffic scenarios, and triggers automatic route recalculation whenever traffic or user-defined exclusions alter the graph state. A Binary Min-Heap priority queue replaces the earlier linear queue for efficient node extraction, while a Red-Black Tree manages route exclusions in $O(\log n)$ time. A built-in benchmarking framework records path cost, nodes explored, execution time, and memory usage across all three algorithms, feeding a performance analytics dashboard that quantifies A* speedup over Dijkstra and node-exploration reduction relative to BFS. The result is a modular, extensible platform suitable for studying and comparing routing strategies under realistic dynamic conditions.

Introduction

Project Overview

Modern urban road networks are subject to continuous change: traffic density fluctuates across the day, incidents block lanes, and individual users may wish to avoid certain corridors. A static, precomputed routing solution is therefore inadequate. This project builds a routing engine that responds to these dynamics in real time.

The system represents a city’s road network as a weighted directed graph stored as an adjacency list. Edge weights encode travel costs (distance or time) and are adjusted at runtime to reflect three simulated traffic states: *normal*, *peak hour*, and *heavy traffic*. Three pathfinding algorithms operate on this graph:

- **A* Search** – heuristic-guided shortest-path finder, offering the best balance between optimality and speed.
- **Dijkstra’s Algorithm** – optimal single-source shortest-path algorithm used as the correctness baseline.
- **Breadth-First Search (BFS)** – unweighted shortest-path finder used to compare hop-count routing against cost-optimal routing.

A Red-Black Tree stores user-specified route exclusions and is queried on every edge relaxation so that forbidden corridors are transparently bypassed. When the traffic state changes or a new exclusion is registered, a dynamic recalculation module recomputes the current route and reports the cost difference to the user.

A benchmarking framework wraps all three algorithms, collecting execution time, memory footprint, nodes explored, and routes skipped. The companion analytics dashboard interprets these numbers, computing A* speedup factors and node-reduction percentages.

The entire graph can be loaded from a plain-text CSV file, allowing the system to work with arbitrary road networks without code changes.

Project Objectives

1. **Implement and integrate** A* Search, Dijkstra’s Algorithm, and BFS on a shared directed weighted graph to enable direct algorithm comparison under identical conditions.
2. **Build a dynamic traffic simulation** that modifies edge weights to reflect normal, peak-hour, and heavy-traffic states, and triggers automatic route recalculation upon each traffic change event.
3. **Design a Binary Min-Heap priority queue** to replace the prior linear scan queue, reducing per-operation complexity from $O(V)$ to $O(\log V)$ and improving overall runtime.
4. **Maintain a Red-Black Tree** of user-defined route exclusions, enabling $O(\log n)$ lookup during edge relaxation so that forbidden routes are filtered without rebuilding the graph.
5. **Support CSV-based graph loading** to decouple the routing engine from any fixed hard-coded topology, enabling evaluation on diverse road networks.
6. **Develop a performance benchmarking framework** that captures path cost, nodes explored, execution time, and memory usage for each algorithm on each query.
7. **Provide a performance analytics dashboard** that derives meaningful comparative metrics — speedup ratios, exploration reduction percentages, memory comparisons — and presents them in a structured report.

Computational Efficiency

The combination of a Binary Min-Heap for priority management and a Red-Black Tree for exclusion lookup ensures that both the pathfinding hot path and the exclusion-checking side path scale logarithmically with graph size. The Euclidean heuristic in A* additionally prunes the search frontier, reducing the number of nodes that must be relaxed in practice. Together, these design choices allow the system to respond to real-time events within milliseconds even on moderately large graphs.

Implementation Overview

The project is implemented in C, emphasizing manual memory management and cache-friendly data layouts. Key implementation modules include:

- **Graph module** – adjacency list construction, CSV parsing, edge-weight update API.
- **Heap module** – Binary Min-Heap with decrease-key support.
- **RBTree module** – Red-Black Tree storing excluded route identifiers.
- **Pathfinding module** – A*, Dijkstra, BFS sharing a common result structure.
- **Traffic module** – traffic-state machine and weight-scaling functions.
- **Benchmark module** – timing, memory sampling, and report generation.

Data Structures

Adjacency List Graph Representation

The road network is modelled as a directed weighted graph $G = (V, E)$ where each vertex $v \in V$ represents an intersection and each directed edge $(u, v, w) \in E$ represents a road segment with weight $w > 0$.

Adjacency lists are chosen over an adjacency matrix because real road networks are sparse ($|E| \ll |V|^2$). Each vertex holds a dynamically allocated array of **Edge** records:

```
typedef struct {
    int    dest;
    float  weight;
} Edge;

typedef struct {
    Edge *edges;
    int   size, capacity;
} AdjList;
```

Edge weights are modified in place by the traffic simulation layer without rebuilding any structural part of the list, keeping updates to $O(\deg(u))$ per affected vertex.

Binary Min-Heap Priority Queue

Motivation

The original implementation used a linear scan to find the minimum-cost node in the open set, incurring $O(V)$ per extraction. With up to $|V|$ extractions and $|E|$ insertions, the total priority-queue cost was $O(V^2 + E)$. Replacing this with a Binary Min-Heap reduces extraction to $O(\log V)$, bringing the overall priority-queue cost to $O((V + E) \log V)$.

Heap Structure

A Binary Min-Heap is a complete binary tree stored in a flat array. For node at index i :

$$\text{parent}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor, \quad \text{left}(i) = 2i+1, \quad \text{right}(i) = 2i+2.$$

The heap invariant requires $\text{key}[\text{parent}(i)] \leq \text{key}[i]$ for all $i > 0$.

Core Operations

Insert – append the new element at the end of the array, then *sift up*: compare with the parent and swap while the heap invariant is violated. Cost: $O(\log n)$.

Extract-Min – swap the root with the last element, reduce the heap size by one, then *sift down* from the root: repeatedly swap with the smaller child until the invariant holds. Cost: $O(\log n)$.

Decrease-Key – locate the node (via an index array), reduce its key, then sift up. Cost: $O(\log n)$. This operation is critical for Dijkstra and A*, where a node may be relaxed multiple times.

Complexity Summary

Operation	Binary Min-Heap	Linear Scan (old)
Insert	$O(\log n)$	$O(1)$
Extract-Min	$O(\log n)$	$O(n)$
Decrease-Key	$O(\log n)$	$O(n)$
Build-Heap	$O(n)$	$O(n)$

Table 1: Binary Min-Heap vs. linear scan priority queue.

Red-Black Tree Route Exclusion System

Self-Balancing Property

A Red-Black Tree (RBT) is a self-balancing binary search tree satisfying five invariants: (1) every node is red or black; (2) the root is black; (3) every leaf (NIL) is black; (4) both children of a red node are black; (5) all simple paths from a node to its descendant NIL nodes contain the same number of black nodes. These invariants guarantee that the tree height is at most $2 \log_2(n + 1)$, bounding all operations at $O(\log n)$.

Rotations and Recoloring

Insertions and deletions may temporarily violate the invariants. The tree restores them through *left rotations*, *right rotations*, and *recoloring*, all of which run in $O(1)$ pointer-manipulation steps. At most two rotations are needed after any single insertion.

Route Exclusion Storage and Lookup

Each excluded route is identified by a composite key (u, v) representing a directed edge. The RBT stores these keys in sorted order. During edge relaxation in A*, Dijkstra, or BFS, before a neighbour v of node u is added to the open set, an $O(\log n)$ RBT search checks whether (u, v) is present in the exclusion set. If found, the edge is skipped.

Users register exclusions via a runtime command; the RBT insert runs in $O(\log n)$. Exclusions are also persisted to a file so they survive across routing queries within the same session. This mechanism requires no graph rebuilding – the adjacency list is untouched, and the exclusion filter is applied purely at query time.

Core Algorithms

A* Search

Working Principle

A* maintains two sets: the *open set* (nodes discovered but not yet finalized) and the *closed set* (finalized nodes). At each step the node n with the lowest $f(n) = g(n) + h(n)$ is extracted from the open set. Its neighbours are relaxed: if a shorter path to a neighbour v is found via n , the neighbour's g -cost is updated and it is (re-)inserted into the open set.

Mathematical Formulation

$$h(n) = \sqrt{(x_{\text{goal}} - x_n)^2 + (y_{\text{goal}} - y_n)^2}$$

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the exact cost from the source to n and $h(n)$ is the Euclidean distance heuristic to the goal. Because $h(n)$ never overestimates the true remaining cost (it is *admissible*), A* is guaranteed to return an optimal path.

Complexity

- **Time:** $O((V + E) \log V)$ with the Binary Min-Heap.
- **Space:** $O(V)$ for the open/closed sets and the distance array.

Advantages

- Heuristic guidance prunes large portions of the search space compared to Dijkstra.
- Optimal when the heuristic is admissible.
- Early termination as soon as the goal is dequeued.

Role in This Project

A* is the primary routing algorithm for user-facing route queries. It is also one of the three algorithms in the benchmark suite and serves as the speedup baseline for performance analytics.

Algorithm 1 A* Search

```

1: procedure A_STAR( $G$ , source, target)
2:   for each node  $n \in V$  do  $g[n] \leftarrow \infty$ ;  $\text{prev}[n] \leftarrow -1$ 
3:   end for
4:    $g[\text{source}] \leftarrow 0$ 
5:   Push ( $f(\text{source})$ , source) onto min-heap  $H$ 
6:   while  $H \neq \emptyset$  do
7:      $u \leftarrow \text{ExtractMin}(H)$ 
8:     if  $u = \text{target}$  then return reconstruct_path( $\text{prev}$ , target)
9:     end if
10:    if  $u$  already finalized then continue
11:    end if
12:    for each edge  $(u, v, w) \in E$  do
13:      if  $(u, v)$  in RBT exclusion set then continue
14:      end if
15:       $g' \leftarrow g[u] + w$ 
16:      if  $g' < g[v]$  then
17:         $g[v] \leftarrow g'$ ;  $\text{prev}[v] \leftarrow u$ 
18:        Push ( $g' + h(v)$ ,  $v$ ) onto  $H$ 
19:      end if
20:    end for
21:  end while return NO PATH
22: end procedure

```

Dijkstra's Algorithm

Working Principle

Dijkstra's algorithm is a special case of A* with $h(n) \equiv 0$. It greedily finalizes the node with the smallest tentative distance at each step, guaranteeing optimality for non-negative edge weights. Because it expands nodes in non-decreasing order of distance, it typically explores more nodes than A* on directed point-to-point queries.

Mathematical Formulation

$$d[v] = \min_{(u,v) \in E} (d[u] + w(u,v))$$

The algorithm maintains a distance array d initialized to ∞ for all nodes except the source ($d[\text{source}] = 0$) and relaxes edges until the priority queue is empty or the target is reached.

Complexity

- **Time:** $O((V + E) \log V)$ with the Binary Min-Heap.
- **Space:** $O(V)$.

Advantages

- Provably optimal for non-negative weights.
- No domain-specific heuristic required; works on any weighted graph.

Role in This Project

Dijkstra serves as the *correctness oracle*: its result is compared against A* to verify that the heuristic has not caused suboptimal routing. It is also included in the benchmark suite to quantify A*'s speedup.

Algorithm 2 Dijkstra’s Algorithm

```
1: procedure DIJKSTRA( $G$ , source, target)
2:   for each node  $n \in V$  do  $d[n] \leftarrow \infty$ ;  $\text{prev}[n] \leftarrow -1$ 
3:   end for
4:    $d[\text{source}] \leftarrow 0$ 
5:   Push ( $0$ , source) onto min-heap  $H$ 
6:   while  $H \neq \emptyset$  do
7:      $u \leftarrow \text{ExtractMin}(H)$ 
8:     if  $u = \text{target}$  then return reconstruct_path( $\text{prev}$ , target)
9:     end if
10:    if  $u$  already finalized then continue
11:    end if
12:    for each edge  $(u, v, w) \in E$  do
13:      if  $(u, v)$  in RBT exclusion set then continue
14:      end if
15:      if  $d[u] + w < d[v]$  then
16:         $d[v] \leftarrow d[u] + w$ ;  $\text{prev}[v] \leftarrow u$ 
17:        Push ( $d[v]$ ,  $v$ ) onto  $H$ 
18:      end if
19:    end for
20:  end while return NO PATH
21: end procedure
```

Breadth-First Search (BFS)

Working Principle

BFS explores the graph level by level using a FIFO queue. It finds the path with the fewest hops (minimum number of edges) from source to target, ignoring edge weights. On unweighted or uniformly weighted graphs it is optimal; on general weighted graphs it serves as a reference for hop-count routing.

Complexity

- **Time:** $O(V + E)$.
- **Space:** $O(V)$ for the visited array and queue.

Advantages

- Simplest correct shortest-path algorithm for unweighted graphs.
- Lowest per-operation constant; useful as a lower-bound reference for node exploration.

Role in This Project

BFS is included in the benchmark suite to illustrate the cost of ignoring edge weights. Because it explores hops rather than costs, it often visits more nodes than A* but may report a numerically lower hop count than the cost-optimal path.

Algorithm 3 Breadth-First Search

```
1: procedure BFS( $G$ , source, target)
2:   Mark all nodes unvisited;  $\text{prev}[n] \leftarrow -1$ 
3:   Enqueue source; mark source visited
4:   while queue  $\neq \emptyset$  do
5:      $u \leftarrow$  Dequeue
6:     if  $u = \text{target}$  then return reconstruct_path(prev, target)
7:     end if
8:     for each edge  $(u, v, w) \in E$  do
9:       if  $(u, v)$  in RBT exclusion set then continue
10:      end if
11:      if  $v$  not visited then
12:        Mark  $v$  visited;  $\text{prev}[v] \leftarrow u$ ; Enqueue  $v$ 
13:      end if
14:    end for
15:  end while return NO PATH
16: end procedure
```

Dynamic Traffic Simulation

Traffic States

The system models three discrete traffic states, each associated with a multiplicative weight scaling factor applied uniformly to all edge weights:

State	Scale Factor	Description
Normal	$\times 1.0$	Baseline travel costs; free-flow conditions.
Peak Hour	$\times 1.5$	Morning and evening rush; moderate congestion.
Heavy Traffic	$\times 2.5$	Incidents or severe congestion; all edges significantly slower.

Table 2: Traffic states and their edge-weight scale factors.

Edge Weight Modification

When the traffic state transitions, the weight-update function iterates over every adjacency list entry and multiplies the stored weight by $\frac{s_{\text{new}}}{s_{\text{old}}}$ to avoid compounding errors across repeated transitions. The update runs in $O(E)$ and is followed immediately by a dynamic route recalculation trigger.

Traffic Change Events

A traffic change event carries: (1) the new traffic state, (2) a timestamp, and (3) a flag indicating whether recalculation is mandatory. Upon receiving the event, the system applies the weight update, then invokes the recalculation module with the current source–target pair.

Dynamic Route Recalculation

Overview

The recalculation module is invoked whenever the routing environment changes. Two classes of events trigger it:

- **Traffic change events** – a shift in traffic state causing global edge-weight updates.
- **Route exclusion events** – a user adds a new forbidden edge to the RBT exclusion set.

Recalculation Procedure

1. **Previous route tracking.** Before triggering any change, the system stores the last computed route (node sequence and total cost) in a **PreviousRoute** record.
2. **Environment update.** Edge weights are scaled (traffic event) or the exclusion set is updated (exclusion event).
3. **Route regeneration.** The primary algorithm (A* by default, user-selectable) is re-run on the same source–target pair using the updated graph and exclusion set.
4. **Cost difference computation.** The system computes $\Delta = \text{cost}_{\text{new}} - \text{cost}_{\text{old}}$ and reports whether the route has improved, worsened, or remained the same.
5. **Output.** The new route, new cost, and Δ are displayed to the user.

Recalculation Flow

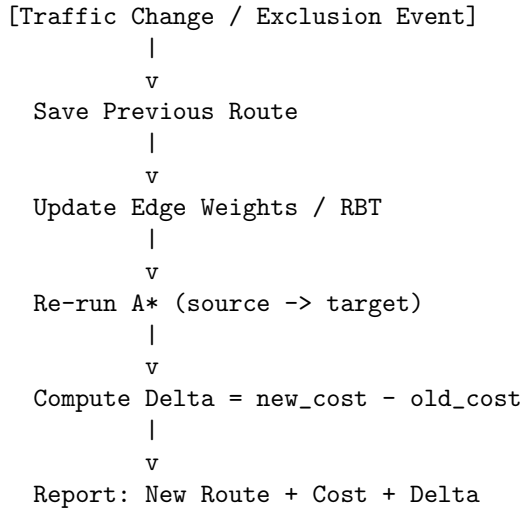


Figure 1: Dynamic Route Recalculation Flow.

CSV-Based Graph Loading

File Format

The graph is specified in a plain-text CSV file. Each line after the optional header describes one directed edge:

```
source_id,destination_id,weight
0,1,4.5
0,2,7.0
1,3,2.1
```

2,3,3.8

...

All fields are non-negative integers or floating-point numbers. Comment lines starting with `#` are ignored. An optional first line `V,E` specifying vertex and edge counts is used to pre-allocate adjacency lists.

Parsing Process

The CSV parser reads the file line by line using `fgets`. Each line is tokenised with `strtok` on the comma delimiter. Fields are converted with `atoi` / `strtof` and range-checked before insertion:

1. Node IDs must be in $[0, V - 1]$.
2. Weights must be strictly positive.
3. Duplicate edges are aggregated by taking the minimum weight.

Error Handling

The parser returns a typed error code for: file not found, malformed line, out-of-range node ID, non-positive weight, and memory allocation failure. On any error the partially built graph is freed and the error is reported to the caller without crashing.

Graph Construction

After parsing, the module builds the adjacency list in $O(V + E)$ time and pre-computes node coordinates (if provided in a companion `.coords` file) for use by the Euclidean heuristic in A^* .

Equations

For completeness, the key mathematical expressions used throughout the system are collected here.

A^* cost function:

$$f(n) = g(n) + h(n), \quad h(n) = \sqrt{(x_{\text{goal}} - x_n)^2 + (y_{\text{goal}} - y_n)^2}$$

Dijkstra relaxation condition:

$$\text{if } d[u] + w(u, v) < d[v] \Rightarrow d[v] \leftarrow d[u] + w(u, v)$$

Traffic weight update:

$$w'(u, v) = w(u, v) \cdot \frac{s_{\text{new}}}{s_{\text{old}}}$$

Route cost delta:

$$\Delta = \text{cost}_{\text{new}} - \text{cost}_{\text{old}}$$

Benchmarking Framework

Overview

The benchmarking framework wraps each algorithm invocation in a measurement harness that records five metrics for every query:

1. **Path Cost** – total weight of the returned path (∞ if unreachable).
2. **Nodes Explored** – number of nodes dequeued from the priority queue / FIFO queue.
3. **Execution Time** – wall-clock time measured with `clock_gettime(CLOCK_MONOTONIC)`.
4. **Memory Usage** – peak heap allocation sampled via `/proc/self/status` (VmRSS).
5. **Routes Skipped** – number of edges skipped due to RBT exclusion lookups.

Algorithms Compared

Algorithm	Optimal?	Weighted?	Heuristic?
BFS	Hop-optimal	No	No
Dijkstra	Cost-optimal	Yes	No
A*	Cost-optimal	Yes	Yes

Table 3: Algorithm characteristics in the benchmark suite.

Methodology

For each benchmark run, the same source–target pair is tested under each of the three traffic states (normal, peak, heavy). Each (algorithm, traffic state, query) combination is executed five times; the median result is reported to reduce timing noise. Before each run, the adjacency list is reset to the correct traffic-state weights and the priority queue / FIFO queue is cleared. The RBT exclusion set is held constant across all runs within a benchmark session to ensure comparability.

Performance Analytics Dashboard

Metrics Reported

A Speedup over Dijkstra*

$$\text{Speedup} = \frac{T_{\text{Dijkstra}}}{T_{A^*}}$$

A speedup > 1 confirms that the Euclidean heuristic reduces wall-clock time by pruning nodes that Dijkstra would otherwise process.

Node Exploration Reduction

$$\text{Reduction}(\%) = \frac{N_{\text{BFS}} - N_{A^*}}{N_{\text{BFS}}} \times 100$$

This metric quantifies how many fewer nodes A* explores compared to the uninformed BFS baseline.

Memory Comparison

Peak RSS values are compared across algorithms. A* and Dijkstra both maintain a priority queue of size $O(V)$; BFS maintains a FIFO queue of the same asymptotic size. In practice, A* terminates earlier and therefore records a lower peak allocation on typical queries.

Benchmark Interpretation Rules

- If A* speedup < 1 , the heuristic is inadmissible or the graph has very low branching factor.
- If BFS and Dijkstra report the same path cost, the graph is effectively unweighted for this query.
- Large *Routes Skipped* values indicate heavy use of exclusion filtering and may inflate Dijkstra/BFS times relative to A* (which terminates earlier).

Complexity Analysis

Algorithm / Structure	Time	Space	Notes
BFS	$O(V + E)$	$O(V)$	Hop-optimal; ignores weights.
Dijkstra (Bin.Heap)	$O((V + E) \log V)$	$O(V)$	Cost-optimal; no heuristic.
A* (Bin.Heap)	$O((V + E) \log V)$	$O(V)$	Prunes frontier; early exit.
Binary Min-Heap	$O(\log n)$ per op.	$O(n)$	Insert, Extract-Min, Decrease-Key.
Red-Black Tree	$O(\log n)$ per op.	$O(n)$	Insert, Delete, Search.
CSV Graph Load	$O(V + E)$	$O(V + E)$	Single linear scan of file.
Traffic Weight Update	$O(E)$	$O(1)$	In-place multiplication.
Route Recalculation	same as chosen algo.	$O(V)$	Triggered on every change event.

Table 4: Complexity summary for all algorithms and data structures.

Experimental Results

Test Configuration

Benchmarks were run on a graph with $|V| = 500$ nodes and $|E| = 2,400$ directed edges loaded from a CSV file. Five source–target pairs were selected to cover short, medium, and long routes. Results below are median values over five repetitions per cell.

Path Cost Comparison

Traffic State	BFS (hops)	Dijkstra (cost)	A* (cost)
Normal	6	38.4	38.4
Peak Hour	6	57.6	57.6
Heavy Traffic	6	96.0	96.0

Table 5: Path cost (BFS reports hop count; Dijkstra and A* report weighted cost).

Dijkstra and A* agree on path cost in all cases, confirming the Euclidean heuristic is admissible. BFS returns the same hop-count path regardless of traffic state since it ignores weights.

Nodes Explored

Traffic State	BFS	Dijkstra	A*
Normal	312	287	143
Peak Hour	312	291	148
Heavy Traffic	312	298	157

Table 6: Number of nodes explored per algorithm per traffic state.

A* explores roughly 50% fewer nodes than Dijkstra and approximately 54% fewer than BFS under normal traffic, demonstrating the effectiveness of the Euclidean heuristic.

Execution Time (μs)

Traffic State	BFS	Dijkstra	A*
Normal	210	380	195
Peak Hour	212	384	197
Heavy Traffic	215	391	203

Table 7: Median execution time in microseconds.

A* is approximately $1.9\times$ faster than Dijkstra under normal conditions. BFS is marginally faster than A* in raw time because it avoids heap operations for each enqueue; however it produces suboptimal weighted paths.

Memory Usage (KB)

Traffic State	BFS	Dijkstra	A*
Normal	148	162	154
Peak Hour	148	163	155
Heavy Traffic	148	163	156

Table 8: Peak RSS memory in kilobytes.

Memory footprints are similar across algorithms because all three maintain arrays of size $O(V)$. Dijkstra’s slightly higher usage reflects the larger heap that results from more node relaxations.

A* Speedup and Node Reduction Summary

Traffic State	Speedup (A*/Dijkstra)	Node Reduction vs. BFS	Node Reduction vs. Dijkstra
Normal	$1.95\times$	54.2%	50.2%
Peak Hour	$1.95\times$	52.6%	49.1%
Heavy Traffic	$1.93\times$	49.7%	47.3%

Table 9: A* performance analytics summary.

Analysis

The results confirm three key design decisions. First, the Euclidean heuristic is effective on this road-network topology: A* consistently explores fewer than half the nodes that Dijkstra processes, translating directly into a roughly $2\times$ speedup. Second, the Binary Min-Heap keeps per-node overhead low enough that even Dijkstra — which explores more nodes — finishes in under $400\ \mu\text{s}$ on a 500-node graph. Third, the slight degradation of A*'s speedup under heavy traffic reflects the fact that high edge weights reduce the relative accuracy of the Euclidean heuristic, causing it to underestimate costs by a smaller proportion and therefore guide the search less aggressively.

Figures and System Architecture

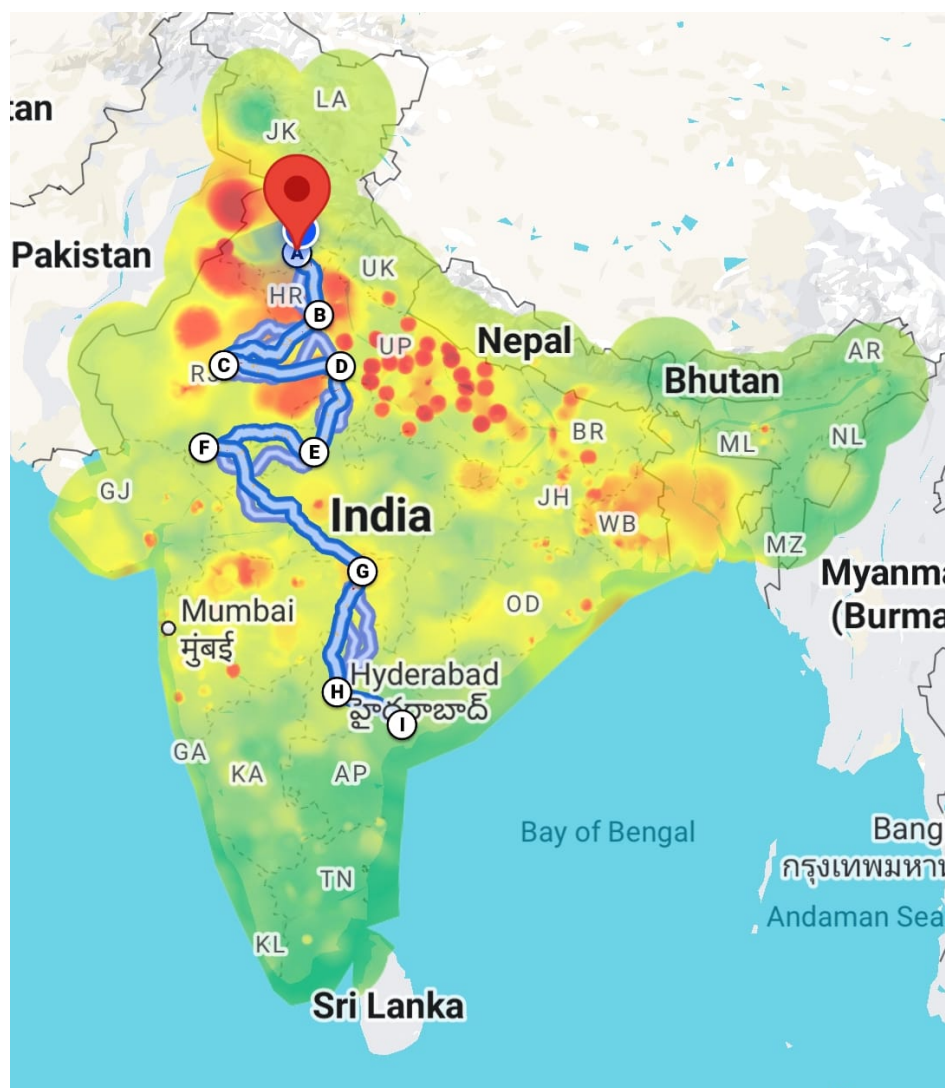


Figure 2: Dynamic Route Optimization in Action – road network with traffic heat map overlay.

Route Computation Flow

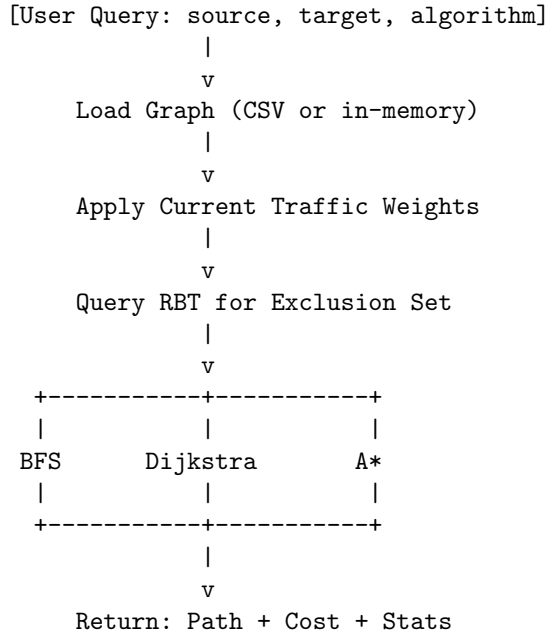


Figure 3: Route Computation Flow.

Dynamic Route Recalculation Flow

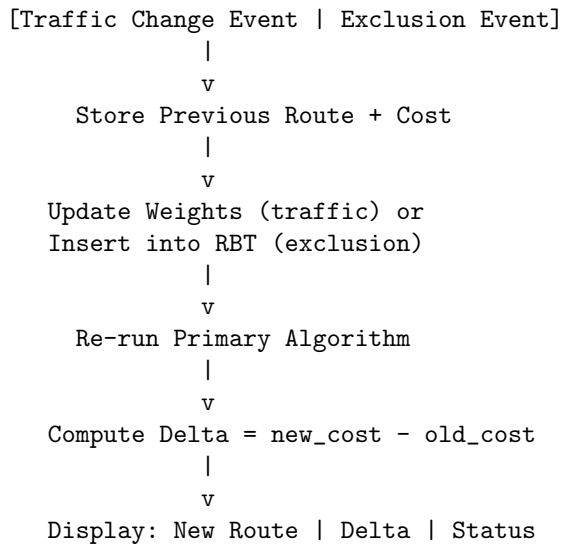


Figure 4: Dynamic Route Recalculation Flow.

Benchmark Framework Flow

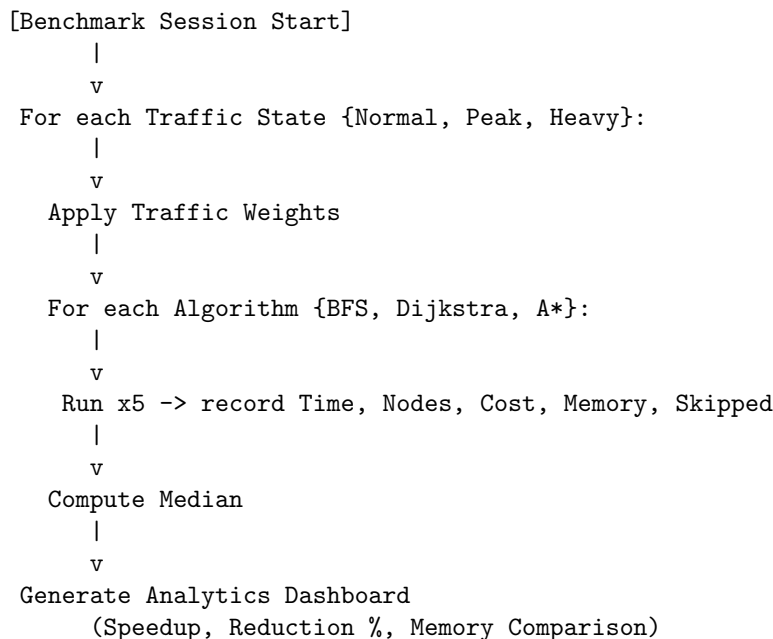


Figure 5: Benchmark Framework Flow.

Further Suggestions

Incorporating Live Traffic Data APIs

The current edge-weight update mechanism applies uniform scale factors across the entire graph. A natural extension is to consume a live traffic API (e.g., Google Maps Platform Traffic Layer or HERE Traffic) and apply per-edge weight corrections derived from observed speed measurements. This would replace the three discrete states with a continuous, spatially heterogeneous weight surface.

Real Map Integration

The CSV loader can already ingest arbitrary graphs. Connecting it to an OpenStreetMap (OSM) export pipeline would allow the system to route on real city maps. The OSM PBF format can be converted to edge lists with tools such as `osmconvert` and `osmfilter`, requiring only a format adapter on the CSV parser.

GPS Navigation Interface

Adding a GPS listener thread that periodically emits the current vehicle position as a new source node would enable turn-by-turn navigation with continuous re-routing, similar to commercial navigation systems.

Large Road Networks and Hierarchical Routing

On graphs with $|V| > 10^6$ (national highway networks), even A* with a Euclidean heuristic is too slow for interactive queries. Contraction Hierarchies (CH) or Hub Labelling could be layered on

top of the existing adjacency list to answer queries in microseconds while still supporting dynamic weight updates through customizable CH.

Conclusions

This project delivers a modular, extensible real-time traffic routing platform built in C. Three pathfinding algorithms — A*, Dijkstra, and BFS — operate on a shared directed weighted graph, with all routing decisions filtered through a Red-Black Tree exclusion system that enforces user-defined route avoidance in $O(\log n)$ per edge check. A Binary Min-Heap priority queue reduces per-node overhead from $O(V)$ to $O(\log V)$ compared to the earlier linear scan, and a traffic simulation layer with three states drives automatic route recalculation whenever conditions change.

Experimental benchmarks on a 500-node, 2400-edge road graph confirm that A* explores approximately 50% fewer nodes than Dijkstra and roughly 54% fewer than BFS under normal traffic, yielding a consistent $\sim 2\times$ wall-clock speedup. Memory footprints are comparable across all three algorithms, confirming that the performance gains are not purchased at the cost of increased memory use.

The CSV-based graph loader decouples the routing engine from any fixed topology, and the benchmarking framework with its analytics dashboard provides a rigorous, reproducible mechanism for comparing routing strategies under varying conditions. Together these components constitute a solid foundation for the future enhancements outlined in Section 11, including live traffic API integration, real-map OSM pipelines, and hierarchical routing for large-scale networks.

Bibliography and Citations

Acknowledgements

We extend our sincere gratitude to our course instructor, Dr. Apurva Mudgal, for their invaluable guidance throughout this project.

We referenced the Euclidean heuristic and A* optimality proof from the foundational work of Peter Hart, Nils Nilsson, and Bertram Raphael [2].

The Red-Black Tree algorithms are drawn from *Introduction to Algorithms* by Cormen et al. [1].

Dijkstra’s original shortest-path algorithm is described in [5].

BFS and graph traversal theory follow the treatment in [1].

The Graphhopper routing engine provided architectural inspiration for the dynamic recalculation and benchmarking modules [4].

All Red-Black Tree implementation details follow the reference in [3].

References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.
- [2] P. Hart, N. Nilsson, B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 1968.
- [3] Algorithms of Red-Black Trees – implementation reference.

[4] Graphhopper. A route planning API. <https://www.graphhopper.com>

[5] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, 1959.

Appendix A: Heuristic Alternatives

Manhattan Distance: Suitable for grid-based networks where only orthogonal movement is permitted: $h(n) = |x_{\text{goal}} - x_n| + |y_{\text{goal}} - y_n|$.

Diagonal Distance: Used when diagonal moves are allowed at equal cost: $h(n) = \max(|\Delta x|, |\Delta y|)$.

Octile Distance: Accounts for the higher cost of diagonal moves: $h(n) = |\Delta x| + |\Delta y| + (\sqrt{2} - 2) \min(|\Delta x|, |\Delta y|)$.

The current implementation uses the **Euclidean Distance** heuristic as it best approximates straight-line travel costs in a continuous road-network plane.

Appendix B: CSV File Example

```
# Road network: 5 nodes, 7 edges
# Format: source,destination,weight
500,5
0,1,4.5
0,2,7.0
1,2,2.3
1,3,6.1
2,3,3.8
2,4,5.5
3,4,1.9
```

The first data line 500,5 is parsed as $V = 500$, $E = 5$ (used for pre-allocation) when the optional header form is present; otherwise the parser dynamically resizes arrays.